

DigitalPulse

統合型心電図解析・患者管理システム

基本設計書

文書番号：DP-BD-001

版数：1.0

作成日：2026年5月5日

作成者：辰田

本書は学術目的で作成されたシステム設計書です

改訂履歴

版数	改訂日	改訂者	改訂内容
1.0	2026年5月5日	辰田	初版作成

目次

改訂履歴	2
目次	3
1. はじめに	4
1.1 本書の目的	4
1.2 適用範囲	4
1.3 用語定義	4
2. システム概要	5
2.1 システムの目的	5
2.2 システム全体像	5
2.3 主要機能サマリ	5
3. システムアーキテクチャ	6
3.1 アーキテクチャ方針	6
3.2 システム構成図	6
3.3 デプロイメント構成	6
3.3.1 フロントエンド (Vercel)	6
3.3.2 バックエンドAPI (Render)	6
3.3.3 AIサービス (Google Cloud Run)	7
3.3.4 データベース (Neon)	7
4. 機能設計	8
4.1 認証・セッション管理機能	8
4.1.1 概要	8
4.1.2 ログインフロー	8
4.1.3 セキュア・レイアウト・パターン	8
4.2 患者管理機能	8
4.2.1 患者一覧表示	8
4.2.2 患者新規登録	8
4.3 心電図解析機能	8
4.3.1 CSVインポート	8
4.3.2 AI推論処理	8
4.3.3 波形可視化	9
4.4 レポート機能	9
4.4.1 PDFレポート出力	9
4.4.2 医師コメント更新	9
5. データ設計	10
5.1 データモデル概要	10
5.2 エンティティ定義	10
5.2.1 users (ユーザーテーブル)	10
5.2.2 patients (患者テーブル)	10
5.2.3 ecg_records (心電図記録テーブル)	11

6. インターフェース設計	12
6.1 REST API一覧	12
6.2 外部サービス連携	12
6.2.1 OpenAI API	12
6.2.2 MIT-BIHデータセット	13
7. 非機能要件	14
7.1 可用性	14
7.2 パフォーマンス	14
7.3 セキュリティ	14
7.4 保守性	14
7.5 拡張性	15
8. 制約・前提条件	16
8.1 技術的制約	16
8.2 ビジネス上の前提条件	16
9. 開発環境・ツール	17
9.1 開発環境一覧	17
10. 課題・リスク管理	18
10.1 既知の課題	18
10.2 リスク一覧	18
11. 承認欄	19

1. はじめに

1.1 本書の目的

本書は、DigitalPulse（統合型心電図解析・患者管理システム）の基本設計を定めた文書である。システム開発に携わる全ての関係者が、システムの目的・構成・機能・非機能要件を共通認識として持つことを目的とする。本書は要件定義書を上位文書とし、詳細設計書の上位文書として位置づけられる。

1.2 適用範囲

本書は以下のシステムコンポーネントに適用される。

- フロントエンドアプリケーション（Next.js 15 / App Router）
- バックエンドAPIサーバー（Spring Boot 3.4 / Java 21）
- AIアナリシスサービス（FastAPI / Python / TensorFlow）
- データベース（PostgreSQL on Neon）
- インフラ・デプロイ環境（Vercel / Render / Google Cloud Run）

1.3 用語定義

用語	説明
ECG	Electrocardiogram（心電図）。心臓の電氣的活動を記録した波形データ
MIT-BIH	Massachusetts Institute of Technology - Beth Israel Hospital 不整脈データベース。心電図AIモデルの学習・検証に使用した公開データセット
SVEB	Supraventricular Ectopic Beat（上室性期外収縮）。心房または房室接合部から発生する異常な心拍
VEB	Ventricular Ectopic Beat（心室性期外収縮）。心室から発生する異常な心拍
Fusion Beat	心室融合不整脈。正常拍と心室性期外収縮が融合した波形
AI推論	学習済みニューラルネットワークモデルを用いて、入力された心電図波形を自動分類する処理
JWT	JSON Web Token。認証トークンの形式
CORS	Cross-Origin Resource Sharing。異なるオリジン間でのHTTPリクエストを許可する仕組み
HttpOnly Cookie	JavaScriptからアクセスできないCookie。XSS攻撃に対するセキュリティ対策

2. システム概要

2.1 システムの目的

DigitalPulseは、医療従事者向けの次世代心電図解析・患者管理プラットフォームである。MIT-BIH不整脈データベースを基にトレーニングされたAIモデルによる自動判定、高精度な波形可視化、および臨床レポート出力機能をセキュアなマルチクラウド環境で提供する。

本システムは以下の社会的課題解決を目的とする。

- 医師の心電図読影業務の負担軽減
- 不整脈の見逃しリスクの低減
- 患者情報と心電図記録の統合管理による診療効率向上
- AIによる所見案生成を通じた電子カルテ記載業務の効率化

2.2 システム全体像

本システムは3層のマイクロサービス構成を採用する。プレゼンテーション層（Next.jsフロントエンド）、アプリケーション層（Spring Boot API + FastAPI AIサービス）、データ層（PostgreSQL データベース）から構成される。

2.3 主要機能サマリ

No.	機能名	概要
F-001	ダッシュボード	リアルタイム統計（総解析数・異常検知数・未確認レポート数）の表示。患者ID・異常フラグによる動的フィルタリング
F-002	患者管理	患者の新規登録（氏名・年齢・性別）、一覧表示、個別患者詳細・検査履歴の参照
F-003	心電図波形解析	Rechartsを用いたインタラクティブ波形表示。パディングトリミングにより心拍を強調表示
F-004	AI自動判定	CNN モデルによる5分類（Normal / SVEB / VEB / Fusion / Unknown）の自動診断
F-005	AI所見生成	OpenAI GPT-4o-miniを活用した電子カルテ向け所見・方針の自動生成
F-006	CSVインポート	MIT-BIH形式（187列）CSVのバルクインポート。患者紐付けオプション付き
F-007	PDFレポート出力	患者情報・波形グラフ・AI診断・医師所見を統合したA4レポートのワンクリック出力
F-008	認証・セッション管理	NextAuthを用いたセッション管理。未認証ユーザーのリダイレクト制御

3. システムアーキテクチャ

3.1 アーキテクチャ方針

本システムは以下のアーキテクチャ方針に基づき設計する。

- マイクロサービスアーキテクチャ：フロントエンド・バックエンドAPI・AIサービスを独立したサービスとして構成し、個別スケーリングを可能にする
- マルチクラウド構成：Vercel（フロントエンド）・Render（バックエンドAPI）・Google Cloud Run（AIサービス）・Neon（データベース）を組み合わせ、高可用性とスケーラビリティを両立する
- API指向設計：REST APIにより各サービス間を疎結合に連携する
- セキュリティバイデザイン：認証・認可・CORS設定・HttpOnly Cookieを設計段階から組み込む

3.2 システム構成図

本システムの論理的な構成は以下の通りである。

層	コンポーネント	デプロイ先
プレゼンテーション層	ecg-frontend (Next.js 15)	Vercel (https://digital-pulse-psi.vercel.app)
アプリケーション層 (API)	ecg-api (Spring Boot 3.4)	Render (https://ecg-backend-api.onrender.com)
アプリケーション層 (AI)	ecg-ai (FastAPI + TensorFlow)	Google Cloud Run (asia-northeast1)
データ層	PostgreSQL 15+	Neon (クラウドマネージド)

3.3 デプロイメント構成

3.3.1 フロントエンド (Vercel)

Next.js 15のApp Routerを採用し、Vercelにデプロイする。Vercelのエッジネットワークにより世界規模での低レイテンシなコンテンツ配信を実現する。GitHubリポジトリと連携し、mainブランチへのプッシュで自動デプロイが行われる。

3.3.2 バックエンドAPI (Render)

Spring Boot 3.4をDockerコンテナとしてRenderにデプロイする。PORT環境変数（デフォルト10000）でポートを制御する。PostgreSQL接続情報はNeonから提供される環境変数（SPRING_DATASOURCE_DRIVER_CLASS_NAME / SPRING_DATASOURCE_PASSWORD / SPRING_DATASOURCE_URL / SPRING_DATASOURCE_USERNAME / SPRING_JPA_DATABASE_PLATFORM）を使用する。

3.3.3 AIサービス (Google Cloud Run)

FastAPIアプリケーションをDockerコンテナとしてGoogle Cloud Runにデプロイする。asia-northeast1リージョン（東京）を使用し、日本国内からのレイテンシを最小化する。学習済みモデル（ecg_model.keras）はコンテナイメージに同梱する。

3.3.4 データベース (Neon)

PostgreSQLをNeonのマネージドサービスとして運用する。接続情報は環境変数としてSpring Bootに渡される。JPA/Hibernateを介してアクセスし、スキーマ管理はddl-auto=noneで手動管理する。

4. 機能設計

4.1 認証・セッション管理機能

4.1.1 概要

NextAuth.jsを使用したセッション管理を採用する。ユーザーはユーザー名とパスワードでログインし、セッションはHttpOnly Cookieで管理される。未認証ユーザーはトップページへ強制リダイレクトされる。

4.1.2 ログインフロー

フロントエンドのNextAuth CredentialsProviderが、入力されたユーザー名・パスワードをバックエンドAPI（POST /api/auth/login）へ送信する。バックエンドはユーザーを検索しパスワードを照合（現時点では平文比較、bcrypt化を予定）する。認証成功時にユーザーIDとユーザー名をレスポンスとして返し、NextAuthがセッションを確立する。

4.1.3 セキュア・レイアウト・パターン

AppLayoutコンポーネントによる番人ロジックを実装する。NextAuthのセッション状態とパス名を監視し、未認証ユーザーをトップページへ強制リダイレクトする。認証時のみサイドバーを表示する条件付きレンダリングを統合し、ログアウト後のサイドバー残存問題を解決する。

4.2 患者管理機能

4.2.1 患者一覧表示

登録済み患者をカード形式で一覧表示する。各カードには氏名・年齢・性別・患者ID・ページネーション（1ページ20件）を表示し、「VIEW HISTORY」ボタンからダッシュボードの該当患者フィルタへ遷移できる。

4.2.2 患者新規登録

登録フォームから氏名・年齢・性別を入力し、バックエンドAPI（POST /api/patients）へ送信する。登録成功後はリストが自動更新される。

4.3 心電図解析機能

4.3.1 CSVインポート

MIT-BIH形式（187列）のCSVファイルをマルチパートフォームデータとして送信する。オプションで患者IDを指定することで、インポートレコードを特定患者に紐付けることができる。インポート処理中にAI推論が実行され、結果がデータベースに保存される。

4.3.2 AI推論処理

バックエンドAPIはAIサービス（POST /api/predict）へ波形データを転送する。AIサービスはCNNモデルで5分類の推論を実行し、クラス名・信頼度・異常フラグを返す。さらにOpenAI GPT-4o-miniで所見テキストを生成し、レポートとして返却する。

4.3.3 波形可視化

Rechartsを使用したAreaChartで心電図波形を表示する。波形データのゼロパディングをトリミングして心拍部分を強調する。異常（isAnomaly=true）時は赤色、正常時は青色でグラデーション表示する。

4.4 レポート機能

4.4.1 PDFレポート出力

html2pdf.jsライブラリを使用し、reportRefのDOM要素をA4 PDFとして出力する。レポートには患者情報・AI診断結果・波形グラフ・医師所見が含まれる。ファイル名はECG_Report_{ID}.pdfとして保存される。

4.4.2 医師コメント更新

解析詳細画面のテキストエリアから医師が所見を修正・追記できる。保存ボタン押下でPUT /api/ecg/{id}/commentを呼び出し、データベースを更新する。

5. データ設計

5.1 データモデル概要

本システムのデータモデルは3つの主要エンティティから構成される。usersテーブル（ユーザー認証情報）、patientsテーブル（患者基本情報）、ecg_recordsテーブル（心電図記録・AI解析結果）である。

5.2 エンティティ定義

5.2.1 users（ユーザーテーブル）

カラム名	型	制約	説明
id	INT	PK, AUTO_INCREMENT	ユーザーID
username	VARCHAR(255)	UNIQUE, NOT NULL	ログインユーザー名
password	VARCHAR(255)	NOT NULL	パスワード（将来的にbcryptハッシュ化）
display_name	VARCHAR(255)	NULL可	表示名
role	VARCHAR(50)	NULL可	ユーザー権限（将来拡張用）

5.2.2 patients（患者テーブル）

カラム名	型	制約	説明
id	INT	PK, AUTO_INCREMENT	患者ID
name	VARCHAR(255)	NOT NULL	患者氏名
age	INT	NULL可	年齢
gender	VARCHAR(50)	NULL可	性別（Male / Female / Other）

5.2.3 ecg_records (心電図記録テーブル)

カラム名	型	制約	説明
id	INT	PK, AUTO_INCREMENT	レコードID
patient_id	INT	FK → patients. id, NULL 可	紐付け患者ID (任意)
recorded_at	DATETIME	NULL可	記録日時 (インポート時刻)
heart_rate	INT	NULL可	心拍数 (将来拡張用)
is_anomaly	TINYINT(1)	NULL可	異常フラグ (true: 異常, false: 正常)
waveform_data	JSON	NOT NULL	187点の波形データ (JSON配列)
doctor_comment	TEXT	NULL可	AI所見 + 医師コメント
diagnosis_type	INT	NULL可	診断分類コード (0:Normal, 1:SVEB, 2:VEB, 3:Fusion, 4:Unknown)

6. インターフェース設計

6.1 REST API一覧

No.	メソッド	エンドポイント	概要
API-001	POST	/api/auth/login	ユーザーログイン認証
API-002	GET	/api/patients	患者一覧取得
API-003	POST	/api/patients	患者新規登録
API-004	GET	/api/patients/{id}	特定患者情報取得
API-005	GET	/api/ecg/all	全心電図レコード取得（ID降順）
API-006	GET	/api/ecg/{id}	特定心電図レコード取得
API-007	GET	/api/ecg/search	患者ID・異常フラグによる絞り込み検索
API-008	PUT	/api/ecg/{id}/comment	医師コメント更新
API-009	POST	/api/ecg/analyze	単一波形のAI解析（リアルタイム）
API-010	POST	/api/ecg/import	CSV一括インポート & AI解析
API-011	POST	/api/ecg/sync-ai	未解析レコードへのAI一括適用（非同期）
AI-001	POST	/api/predict（AIサービス）	波形データの5分類推論 + 所見生成
AI-002	GET	/health（AIサービス）	AIサービスヘルスチェック

6.2 外部サービス連携

6.2.1 OpenAI API

AIサービスはOpenAI APIのgpt-4o-miniモデルを使用して所見テキストを生成する。APIキーはCloud Runの環境変数（OPENAI_API_KEY）として管理し、.envファイルからロードする。temperature=0.3で応答の安定性を確保する。

6.2.2 MIT-BIHデータセット

心電図AIモデルの学習はMIT-BIHデータベースの公開データを使用した。データは187点の時系列データとして正規化されており、本システムでも同形式のCSVを入力として要求する。

7. 非機能要件

7.1 可用性

- ・ フロントエンド (Vercel) : Vercel SLAに準拠し、99.99%以上の稼働率を目標とする
- ・ バックエンドAPI (Render) : 無料プランでは非アクティブ時にスリープするため、初回レスポンスに最大30秒の遅延が生じる場合がある
- ・ AIサービス (Cloud Run) : コンテナの自動スケールアウトにより需要に応じた可用性を確保する

7.2 パフォーマンス

- ・ API応答時間: 通常リクエストは1秒以内を目標とする
ページネーションによる定数時間での応答 (件数に依存しない速度)
- ・ AI推論時間: 単一波形のCNN推論は100ms以内を目標とする。OpenAI API呼び出しを含む総処理時間は5秒以内を目標とする
- ・ CSV一括インポート: 100行のCSVを60秒以内に処理完了することを目標とする
- ・ 起動高速化: PostgreSQLDialectの明示指定・JDBCメタデータ取得スキップ・Hibernateロギングレベル削減により起動時間を最適化する

7.3 セキュリティ

- ・ 認証: NextAuth.jsによるセッション管理とHTTPOnly Cookieでセッション情報を保護する
- ・ CORS: Spring Boot側でallowedOriginPatternsを厳密に設定し、正規ドメインのみからのリクエストを受け付ける (credentials: include対応)
- ・ CSRF: Spring SecurityのCSRF保護を一時的に無効化しているが、将来的にSameSite=StrictまたはCSRFトークンで保護する計画である
- ・ パスワード: 現時点では平文比較を採用しているが、将来的にbcryptによるハッシュ化を実施する
- ・ APIキー: OPENAI_API_KEYはCloud Runの環境変数として安全に管理し、ソースコードには含めない

7.4 保守性

- ・ コード品質: Lombokを活用し boilerplate を削減する
- ・ 依存関係: Mavenとnpmによるパッケージ管理を行い、依存ライブラリのバージョンを固定する
- ・ 環境分離: application.propertiesでプロファイルごとの設定を管理し、本番・開発環境を分離する

7.5 拡張性

- AI分類の拡張：CLASS_NAMESディクショナリへの追加により分類クラスを容易に拡張できる
- 認証の強化：Spring Security + JWT方式への移行を将来的に計画する
- モニタリング：Cloud RunのCloud Monitoringと連携したアラート設定が可能である

8. 制約・前提条件

8.1 技術的制約

- AIモデルの入力形式：心電図データは187点のFloat配列形式でなければならない（MIT-BIH準拠）
- ブラウザサポート：モダンブラウザ（Chrome / Firefox / Safari 最新版）を対象とする
- 接続環境：インターネット接続が必要（オフライン動作は非対応）
- データベース：PostgreSQLとの互換性を前提とする（H2はテスト環境のみ）

8.2 ビジネス上の前提条件

- 本システムは学術目的で作成されており、実際の医療診断に使用するためには医療機器認証が別途必要となる
- OpenAI APIの利用料金が発生するため、商用利用前にコスト計画を策定する必要がある
- MIT-BIHデータベースのライセンス条件に準拠した利用に限定される

9. 開発環境・ツール

9.1 開発環境一覧

区分	ツール・技術	バージョン / 備考
フロントエンドFW	Next.js	15.x / App Router
バックエンドFW	Spring Boot	3.4.x / Java 21
AIサービスFW	FastAPI	最新版 / Python 3.11
AI学習FW	TensorFlow / Keras	2.15.0
LLM API	OpenAI API (gpt-4o-mini)	openai Python SDK
ORMマッパー	Spring Data JPA / Hibernate	Spring Boot管理
認証ライブラリ	NextAuth.js	最新版
チャートライブラリ	Recharts	最新版
PDF生成	html2pdf.js	最新版
CSVパース	Apache Commons CSV	1.10.0
ビルドツール (BE)	Maven	mvnw (ラッパー)
コンテナ	Docker	各サービスにDockerfile有
バージョン管理	Git / GitHub	mainブランチ運用
IDE	VSCode	.vscode設定含む

10. 課題・リスク管理

10.1 既知の課題

No.	課題内容	優先度	対応方針
IQ-001	パスワードが平文で比較・保存されている	高（セキュリティ）	bcryptによるハッシュ化を実装する
IQ-002	CSRF保護が無効化されている	高（セキュリティ）	SameSite=Strict CookieまたはCSRFトークンを導入する
IQ-003	Renderの無料プランでコールドスタート遅延が発生する	中（パフォーマンス）	定期的なヘルスチェックリクエストによるウォームアップ、または有料プランへの移行を検討する
IQ-004	AIサービスのCORSがallow_origins=["*"]で全オリジン許可	中（セキュリティ）	許可オリジンをフロントエンドドメインに限定する
IQ-005	waveform_dataのJSON列がフルスキャンになる可能性	低（パフォーマンス）	is_anomaly・patient_idへのインデックス設定で検索を最適化する

10.2 リスク一覧

No.	リスク内容	発生可能性	軽減策
RK-001	OpenAI APIの料金超過	中	API使用量の上限設定とコストアラートを設定する
RK-002	NeonのDB接続情報変更による障害	低	環境変数管理を徹底し、接続文字列のハードコードを排除する
RK-003	AIモデルの精度がデプロイ後に低下	低	定期的なモデル再評価と再トレーニングのプロセスを整備する

11. 承認欄

役割	氏名	署名	承認日
作成者	辰田		2026年5月5日
レビュー者			
承認者			

以上